



MSE 153: Introduction to IT

Binary Number System



Lecture Outline

- Numbering Systems
- Binary Numbering Systems
- Negative Number and Fractions
- Character Representations
- Binary Operations
- Examples of Using Binary Numbers
 - Network Address and Netmasks
 - Image Files
 - Error Detection and Correction



The Binary Numbering Systems

- A numbering system (base) is a way to represent numbers, base k
 - We denote the base by adding k as a subscript at the end of the number as in 1234_5 for base 5 (we can omit 10 if in base 10)
- Decimal is base 10, binary is base 2
 - We use 10 because of 10 fingers, but are interested in 2 (also 8 and 16) because computers store and process information in a digital (on/off) way
- 1 binary digit is a bit
- In 1 bit, we store a 0 or a 1
 - This doesn't give us much meaning, just 1/0, yes/no, true/false
- We group 8 bits together to store 1 byte
 - 00000000 to 11111111
 - In 1 byte, we can store a number from 0 to 255 or a character (e.g., 'a', '\$', '8', ' ')



Interpreting Numbers

- The base tells us how to interpret each digit
 - The column that the digit is in represents the value base column
 - the rightmost column is always column 0
 - Example:
 - 5372 in base 10 has
 - 5 in the 10^3 column (1,000)
 - 3 in the 10^2 column (100)
 - 7 in the 10^1 column (10)
 - 2 in the 10^0 column (1)
- To convert from some base k to base 10, apply this formula
 - $abcde_k = a * k^4 + b * k^3 + c * k^2 + d * k^1 + e * k^0$
 - a, b, c, d, e are the digits, k^0 is always 1
- To convert binary to decimal, the values of k^0, k^1, k^2 , etc are powers of 2 (1, 2, 4, 8, 16, 32, ...)



Binary to Decimal Conversion

- Multiply each binary bit by its column value
 - In binary, our columns are (from right to left)
 - $2^0 = 1$
 - $2^1 = 2$
 - $2^2 = 4$
 - $2^3 = 8$
 - $2^4 = 16$
 - $2^5 = 32$
 - Etc
 - $10110 = 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 0 * 2^0 = 16 + 0 + 4 + 2 + 0 = 22$
 - $1100001 = 1 * 2^6 + 1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0 = 64 + 32 + 0 + 0 + 0 + 0 + 1 = 97$



Simplifying Conversion in Binary

- Our digits will either be 0 or 1
 - $0 * \text{anything}$ is 0
 - $1 * \text{anything}$ is that thing
- Just add together the powers of 2 whose corresponding digits are 1 and ignore any digits of 0
- $10110 = 2^4 + 2^2 + 2^1 = 16 + 4 + 2 = 22$
- $1100001 = 2^6 + 2^5 + 2^0 = 64 + 32 + 1 = 97$



Examples

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
0	1	0	1	0	1	0	1	
	64		16		4		1	= 85

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
1	1	1	1	1	0	1	1	
128	64	32	16	8		2	1	= 251

$$11010110 =$$

$$128 + 64 + 16 + 4 + 2$$

$$= 214$$

$$10001011 = 128 + 8 + 2 + 1$$

$$= 139$$

$$11111111 = 128 + 64 + 32 +$$

$$16 + 8 + 4 + 2 + 1$$

$$= 255$$

$$00110011 = 32 + 16 +$$

$$2 + 1$$

$$= 51$$



Converting from Decimal to Binary

- The typical approach is to continually divide the decimal value by 2, recording the quotient and the remainder until the quotient is 0
- The binary number is the group of remainder bits written in opposite order
 - Convert 19 to binary
 - $19 / 2 = 9$ remainder 1
 - $9 / 2 = 4$ remainder 1
 - $4 / 2 = 2$ remainder 0
 - $2 / 2 = 1$ remainder 0
 - $1 / 2 = 0$ remainder 1
 - $19 = 10011_2$



Record the remainders
and then write them
in opposite order



Examples

Convert 200 to binary

$$200 / 2 = 100 \text{ r } 0$$

$$100 / 2 = 50 \text{ r } 0$$

$$50 / 2 = 25 \text{ r } 0$$

$$25 / 2 = 12 \text{ r } 1$$

$$12 / 2 = 6 \text{ r } 0$$

$$6 / 2 = 3 \text{ r } 0$$

$$3 / 2 = 1 \text{ r } 1$$

$$1 / 2 = 0 \text{ r } 1$$

$$200 = 11001000$$

Convert 16 to binary

$$16 / 2 = 8 \text{ r } 0$$

$$8 / 2 = 4 \text{ r } 0$$

$$4 / 2 = 2 \text{ r } 0$$

$$2 / 2 = 1 \text{ r } 0$$

$$1 / 2 = 0 \text{ r } 1$$

$$16 = 10000$$

Convert 21 to binary

$$21 / 2 = 10 \text{ r } 1$$

$$10 / 2 = 5 \text{ r } 0$$

$$5 / 2 = 2 \text{ r } 1$$

$$2 / 2 = 1 \text{ r } 0$$

$$1 / 2 = 0 \text{ r } 1$$

$$21 = 10101$$

Convert 122 to binary

$$122 / 2 = 61 \text{ r } 0$$

$$61 / 2 = 30 \text{ r } 1$$

$$30 / 2 = 15 \text{ r } 0$$

$$15 / 2 = 7 \text{ r } 1$$

$$7 / 2 = 3 \text{ r } 1$$

$$3 / 2 = 1 \text{ r } 1$$

$$1 / 2 = 0 \text{ r } 1$$

$$122 = 1111010$$



Another Technique

- Recall to convert from binary to decimal, we add the powers of 2 for each digit that is a 1
- To convert from decimal to binary, we can subtract all of the powers of 2 that make up the number and record 1s in corresponding columns
- Example
 - $19 = 16 + 2 + 1$
 - So there is a 16 (2^4), a 2 (2^1) and 1 (2^0)
 - Put 1s in the 4th, 1st, and 0th columns:
 - $19 = 10011_2$



Examples

- Convert 122 to binary
 - Largest power of $2 \leq 122 = 64$ leaving $122 - 64 = 58$
 - Largest power of $2 \leq 58 = 32$ leaving $58 - 32 = 26$
 - Largest power of $2 \leq 26 = 16$ leaving $26 - 16 = 10$
 - Largest power of $2 \leq 10 = 8$ leaving $10 - 8 = 2$
 - Largest power of $2 \leq 2 = 2$ leaving $2 - 2 = 0$
 - Done
- $122 = 64 + 32 + 16 + 8 + 2 = 1111010$
- More examples:
 - $555 = 512 + 32 + 8 + 2 + 1 = 1000101011$
 - $200 = 128 + 64 + 8 = 11001000$
 - $199 = 128 + 64 + 4 + 2 + 1 = 11000111$
 - $31 = 16 + 8 + 4 + 2 + 1 = 11111$
 - $60 = 32 + 16 + 8 + 4 = 111100$
 - $1000 = 512 + 256 + 128 + 64 + 32 + 8 = 1111101000$
 - $20 = 16 + 4 = 10100$



Number of Bits

- Notice in our previous examples that for 555 we needed 10 bits and for 25 we only needed 5 bits
- The number of bits available tells us the range of values we can store
- In 8 bits (1 byte), we can store between 0 and 255
 - $00000000 = 0$
 - $11111111 = 255$ ($128 + 64 + 32 + 16 + 8 + 4 + 2 + 1$)
- In n bits, you can store a number from 0 to $2^n - 1$
 - For 8 bits, $2^8 = 256$, the largest value that can be stored in 8 bits is 255
 - What about 5 bits?
 - What about 3 bits?



Negative Numbers

- To store negative numbers, we need a bit to indicate the sign
 - 0 = positive, 1 = negative i.e using a sign bit e.g. -5 would be written as 10000101
- Several representations for negative numbers, we use two's complement
- Positive numbers are the same as in our previous approach
- Negative numbers need to be converted, two ways to do this:

- NOT (flip) all of the bits (1 becomes 0, 0 becomes 1)
- Add 1
- Example: -57 in 8 bits
 - $+57 = 32 + 16 + 8 + 1 = 00111001$
 - $-57 = \text{NOT}(00111001) + 1 = 11000110 + 1 = 11000111$
- Starting from the right of the number
 - record each bit THROUGH the first 1
 - flip all of the remaining bits
 - 1s become 0s, 0s become 1s

Negative (Fractional) Powers of 2

Exponential Form	Value as a Fraction	Value as a Real Number
$2^{-1} = 1/2^1$	1/2	0.5
$2^{-2} = 1/2^2$	1/4	0.25
$2^{-3} = 1/2^3$	1/8	0.125
$2^{-4} = 1/2^4$	1/16	0.0625
$2^{-5} = 1/2^5$	1/32	0.03125
$2^{-6} = 1/2^6$	1/64	0.015625
$2^{-7} = 1/2^7$	1/128	0.0078125



Examples (all are 8 bits)

- -57

- $+57 = 00111001$
- from the right, copy all digits through the first one:
- -----1
- Flip remaining bits (0011100)
- $1100011 \ 1 = 11000111$

- -108

- $+108 = 01101100$
- from the right, copy all digits through the first one:
- -----100
- flip the rest of the bits (01101)
- $10010 \ 100 = 10010100$

- -96

- $+96 = 01100000$
- from right, copy all bits through the first one:
- --100000
- flip rest of the bits (01)
- $10 \ 100000 = 10100000$

- -5

- $+5 = 00000101$
- from right, copy all bits through the first one:
- -----1
- flip rest of the bits (0000010)
- $1111101 \ 1 = 11111011$

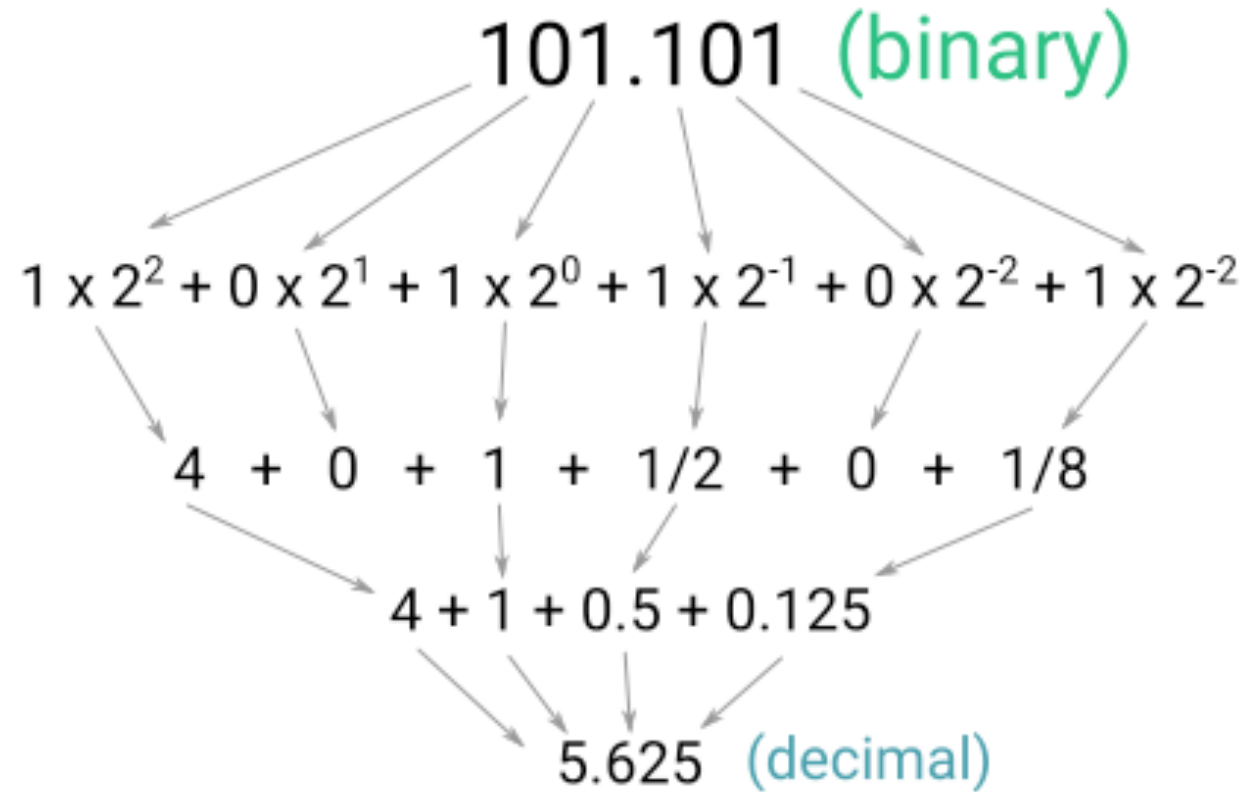


Real (Fractional) Numbers

- Extend our powers of two to the right of the decimal point using negative powers of 2
- $101.1 = 2^2 + 2^0 + 2^{-1}$
 - What is 2^{-1} ? $1/2^1$
 - $2^{-2} = 1/2^2 = 1/4$, $2^{-3} = 1/2^3 = 1/8$
 - $10110.101 = 16 + 4 + 2 + 1/2 + 1/8 = 22.625$
- How do we represent the decimal point?
 - We use a floating point representation, like scientific notation, but in binary where we store 3 integer numbers, a sign bit (1 = negative, 0 = positive), the mantissa (the number without a decimal point) and the location of the decimal point as an exponent
 - The point is that we do not try to store a decimal point, but we record where the decimal point should be
 - For example : $110111.01 = .11011101 * 2^6$
 - Mantissa = 11011101
 - Exponent = 110 (6, i.e. convert the exponent to binary)
 - Sign = 0
 - We simply store this as 011011011101



Real (Fractional) Numbers





Representing decimal points

Single precision, which uses 32 bits and has the following layout:

- **1 bit** for the sign of the number. 0 means positive and 1 means negative.
- **8 bits** for the exponent.
- **23 bits** for the mantissa.

Single Precision Floating Point

1 10110110 11000101100111010110111

↑ ↑ ↑

Sign Exponent Mantissa

$$-6.384063 \times 10^{16}$$



Character Representations

- We need to invent a way to represent or store letters of the alphabet (there is no natural way)
 - Need to differentiate between upper and lower case letters – so we need at least 52 representations
 - We will want to also represent punctuation marks
 - Also digits (phone numbers use numbers but are not stored numerically)
- 3 character codes have been developed
 - EBCDIC – used only by IBM mainframes
 - (Extended Binary Coded Decimal Interchange Code)
- ASCII (American Standard Code for Information interchange)
the most common code, 7 bits – 128 different characters (add a 0 to the front to make it 8 bits or 1 byte per character)
 - Unicode – expands ASCII to 16 bits to represent over 65,000 characters



First 32 characters are control characters (not printable)

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	\$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	



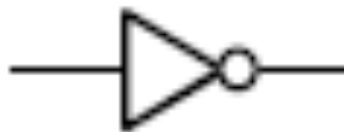
Example

- To store the word “Hello”
 - H = 72 = 01001000
 - e = 101 = 01100101
 - l = 108 = 01101100
 - l = 108 = 01101100
 - o = 111 = 01101111
- Hello = 01001000 01100101 01101100 01101100 01101111
- How much storage space is required for the string
 - R U 4 Luv?
- 10 bytes (5 letters, 3 spaces, 1 digit, 1 punctuation mark)
 - The ‘U’ and ‘u’ are represented using different values
 - ‘U’ = 01010101
 - ‘u’ = 01110101
 - The only difference between an upper and lower case letter is the 3rd bit from the left
 - upper case = 0, lower case = 1



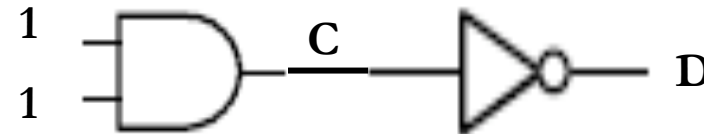
Logical Operators: AND, OR, XOR, NOT Gates

AND 

NOT 

Try this

OR 



XOR 

Output at C and D



Binary Operations

- We learn the binary operations using truth tables

X	Y	AND
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	OR
0	0	0
0	1	1
1	0	1
1	1	1

X	NOT
0	1
1	0

X	Y	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- Given two bits, apply the operator
 - $1 \text{ AND } 0 = 0$
 - $1 \text{ OR } 0 = 1$
 - $1 \text{ XOR } 0 = 1$
- Apply the binary (Boolean) operators bitwise (in columns) to binary numbers as in
 - $10010011 \text{ AND } 00001111 = 00000011$



Examples

- AND – if both bits are 1 the result is 1, otherwise 0
 - $11111101 \text{ AND } 00001111 = 00001101$
 - $01010101 \text{ AND } 10101010 = 00000000$
 - $00001111 \text{ AND } 00110011 = 00000011$
- OR – if either bit is 1 the result is 1, otherwise 0
 - $10101010 \text{ OR } 11100011 = 11101011$
 - $01010101 \text{ OR } 10101010 = 11111111$
 - $00001111 \text{ OR } 00110011 = 00111111$
- NOT – flip (negate) each bit
 - $\text{NOT } 10101011 = 01010100$
 - $\text{NOT } 00001111 = 11110000$
- XOR – if the bits differ the result is 1, otherwise 0
 - $10111100 \text{ XOR } 11110101 = 01001001$
 - $11110000 \text{ XOR } 00010001 = 11100001$
 - $01010101 \text{ XOR } 01011110 = 00001011$



Binary Addition

- To add 2 bits, there are four possibilities
 - $0 + 0 = 0$
 - $1 + 0 = 1$
 - $0 + 1 = 1$
 - $1 + 1 = 2$ – we can't write 2 in binary, but 2 is 10 in binary, so write a 0 and carry a 1
- To compute anything useful (more than 2 single bits), we need to add binary numbers
- This requires that we chain together “carrys”
 - The carry out of one column becomes a carry in the column to its left



Continued

- With 3 bits (the two bits plus the carry), we have 4 possibilities:
 - $0 + 0 + 0 = 0$
 - 2 zeroes and 1 one = 1
 - 2 ones and 1 zero = 2 (carry of 1, sum of 0)
 - 3 ones = 3 (carry of 1 and sum of 1)

- Example:

Carry:		1	1	0	0	Initial carry
X:		0	1	1	1	in is 0
Y:	+	0	1	1	0	
Sum:		1	1	0	1	
			Carry	Carry	No Carry	

Check your work, convert to decimal!



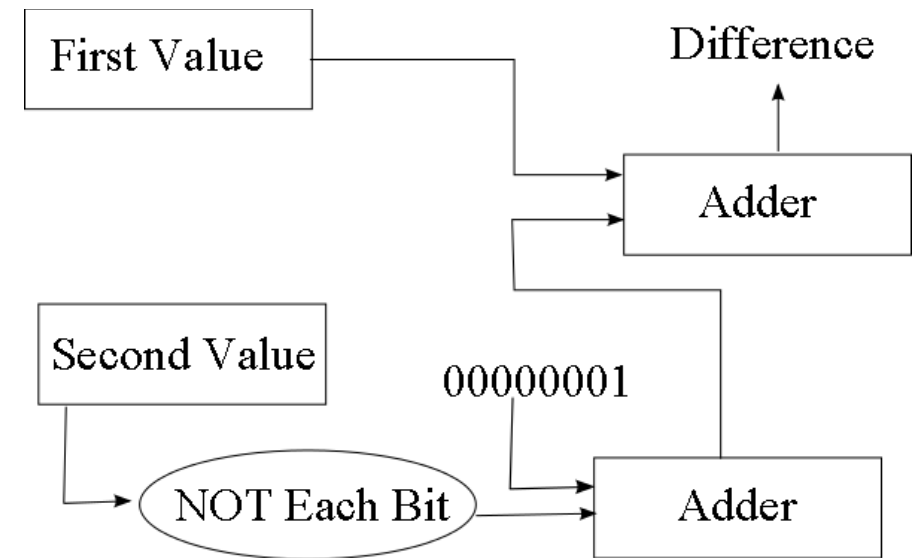
Addition Using AND, OR, XOR

- To implement addition in the computer, convert addition to AND, OR, NOT and XOR
- Input for any single addition is two binary numbers and the carry in from the previous (to the right) column
 - For column i , we will call these X_i , Y_i and C_i
- Compute sum and carry out for column i (S_i , C_{i+1})
- $S_i = (X_i \text{ XOR } Y_i) \text{ XOR } C_i$
 - Example: if $1 + 0$ and carry in of 1
 - $\text{sum} = (1 \text{ XOR } 0) \text{ XOR } 1 = 1 \text{ XOR } 1 = 0$
- $C_{i+1} = (X_i \text{ AND } Y_i) \text{ OR } (X_i \text{ AND } C_i) \text{ OR } (Y_i \text{ AND } C_i)$
 - Example: if $1 + 0$ and carry in of 1
 - $\text{carry out} = (1 \text{ AND } 0) \text{ OR } (1 \text{ AND } 1) \text{ OR } (0 \text{ AND } 1) = 0 \text{ OR } 1 \text{ OR } 0 = 1$
- Try it out on the previous problem



Subtraction

- From math, $A - B = A + (-B)$
- Store A and B in two's complement
- We can convert B into $-B$ by
 - Flip all bits in B and adding 1
 - to flip all bits, apply NOT to each bit
 - to add 1, add the result to 00000001
- We build a subtracter unit to perform this





Network Addresses

- Internet Protocol (IP) version 4 uses 32-bit addresses comprised of 4 octets
 - 1 octet = 8 bits (0..255)
 - Each octet is separated by a period
- The address 10.251.136.253
 - Stored as 00001010.11111011.10001000.11111101 in binary
 - Omit the periods when storing the address in the computer
- The network address comprises two parts
 - The network number (net. id)
 - The machine number on the network (host id)
- The number of bits used for the network number differs depending upon the class of network
 - We might have a network address as the first 3 octets and the machine number as the last octet
 - The netmask is used to return either the network number or the machine number



Network Addresses: IP Classful Addressing

IP addressing supports five different network classes:

- **Class A** : Mainly used with a few very large networks. Provide only 7 bits for the network address field.
- **Class B** : 14 bits for the network address field, 16 bits for the host address field. Offers a good compromise between network and host address space
- **Class C** : Allocate 21 bits for the network address field, provide only 8 bits for the host field. The number of hosts may be a limiting factor
- **Class D** : Reserved for multicast groups, the 4 highest-order bits are set to 1, 1, 1, and 0.
- **Class E** : Reserved for future use, the four highest-order bits are all set to 1.



Netmask Example

- If our network address is the first 3 octets, our network netmask is 255.255.255.0
 - 11111111.11111111.11111111.00000000
- AND this to your IP address 10.251.136.253
 - 11111111.11111111.11111111.00000000
 - AND 00001010.11111011.10001000.11111101
- Gives 00001010.11111011.10001000.00000000
 - or 10.251.136.0 which is the network number
- The machine number netmask is 0.0.0.255
 - What value would you get when ANDing 10.251.136.253 and 0.0.0.255?



Another Example

- In this case, the network address is the first 23 bits (not 24)
- The netmask for the network is 255.255.240.0

IP Address: 00001010 . 11111011 . 10001000 . 11111101 (10.251.136.253)

Netmask: 11111111 . 11111111 . 11110000 . 00000000 (255.255.240.0)

Network

Address: 00001010 . 11111011 . 10000000 . 00000000 (10.251.128.0)

Try some exercises on your own



Image Files

- Any image is a collection of points of light.
- Images stored as sequences of pixels (picture elements)
 - row by row, each pixel is denoted by a value
- A 1000 x 1000 pixel image requires 1,000,000 bits. This would be 125,000 bytes, or roughly 125 KB.
- A 1024x1024 pixel image will comprise 1024 individual dots in one row for 1024 rows (1M pixels)
- This file is known as a bitmap
- In a black and white bitmap, we can store whether a pixel is white or black with 1 bit
 - The 1024x1024 image takes 1Mbit (1 megabit)
- A color image is stored using red, green and blue values
 - Each can be between 0 and 255 (8 bits)
 - So each pixel takes 3 bytes
 - The 1024x1024 image takes 3MBytes
- JPG format discards some detail to reduce the image's size to about 1MB using lossy compression, GIF format uses a standard palette of colors to reduce size from 3 bytes/pixel to 1 (lossless compression)



Parity Check

- Parity bit appended to a block of data
- Even parity
 - Added bit ensures an even number of 1s
- Odd parity
 - Added bit ensures an odd number of 1s
- Example, 7-bit character [1110001]
 - Even parity [11100010];
- Odd parity [11100011]
 - If 10100011 received, there is an error.